



Université Gaston Berger de Saint-Louis
Institut Polytechnique de Saint-Louis
GeIT 2

Système de Gestion des Notes (SGN) :
Analyse Lexicale et Syntaxique avec Flex, Bison

Réalisés par :

Papa Mbaye BA

Gilbert Ngor Thiomby DIOUF

Enseignant :

Dr El Hadji Malick NDOYE

Date de Soumission : 15 Juin 2026

Table des matières

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Objectifs pédagogiques	2
1.3	Flux global d'exécution	2
2	Spécification du langage .sgn	2
2.1	Description du domaine	2
2.2	Structure d'un fichier .sgn	3
2.3	Grammaire formelle (BNF)	3
2.4	Tableau des tokens lexicaux	4
3	Analyseur lexical — <code>lexer.1</code> (Flex)	4
3.1	Structure du fichier	4
3.2	Options et définitions de macros	5
3.3	Règles de reconnaissance	5
3.4	Gestion des erreurs lexicales	6
4	Analyseur syntaxique et sémantique — <code>parser.y</code> (Bison)	6
4.1	Déclarations et union de valeurs	6
4.2	Variables de contexte	7
4.3	Règles de production principales	7
4.4	Les quatre vérifications sémantiques	8
4.5	Organigramme du traitement d'une erreur sémantique	8
5	Implémentation C — <code>main.c</code> et <code>symboles.h</code>	8
5.1	Architecture des fichiers source	8
5.2	Structures de données	8
5.3	Variables globales	10
5.4	Catalogue des fonctions implémentées	10
5.5	Calculs — formules du sujet	10
5.5.1	Moyenne pondérée d'un semestre (formule 1)	10
5.5.2	Moyenne annuelle (formule 2)	11
5.5.3	Décision et mention	11
5.5.4	Tri et calcul des rangs	11
5.6	Organigramme de <code>main()</code>	12
5.7	Sérialisation JSON	12
6	Compilation — <code>Makefile</code>	13
6.1	Compatibilité multi-plateformes	13
6.2	Chaîne de compilation	14
7	Interface graphique Python	14
7.1	Architecture des fichiers	14
7.2	Diagramme de séquence : analyse d'un fichier	15
7.3	Communication subprocess	15

7.4	Fonctionnalités de l'interface	15
8	Tests et validation	16
8.1	Stratégie de tests	16
8.2	Vérification de la compilation	16
8.3	Résultats obtenus	16
8.3.1	Test 1 — Fichier valide	17
8.3.2	Test 4 — Erreurs sémantiques	18
9	Conclusion	18
9.1	Bilan des fonctionnalités	19
9.2	Difficultés rencontrées	19
9.3	Perspectives d'amélioration	19

1. Introduction

1.1. Contexte et motivation

Ce projet s'inscrit dans le module de **Compilation**. Il consiste à concevoir et implémenter un **Système de Gestion des Notes (SGN)** pour les trois premières années de Licence (L1, L2, L3), en mettant en œuvre l'ensemble de la chaîne de traitement d'un langage dédié : de la grammaire formelle jusqu'à l'interface graphique.

1.2. Objectifs pédagogiques

Le projet couvre les compétences suivantes :

- Conception d'une **grammaire formelle BNF** pour un langage de description de données
- Implémentation d'un **analyseur lexical** avec Flex 2.6
- Implémentation d'un **analyseur syntaxique et sémantique** avec Bison 3.x (LALR-1)
- Programmation en **C (C99)** pour les structures de données et la sérialisation JSON
- Développement d'une **interface graphique** Python/PyQt6 communiquant avec le binaire C via `subprocess`

1.3. Flux global d'exécution

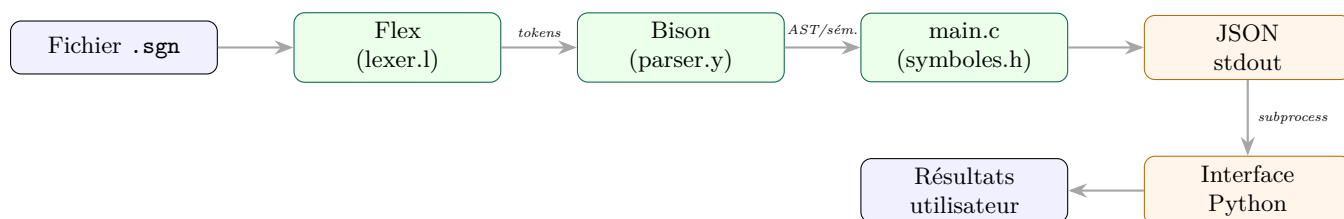


FIG. 1 : Flux global d'exécution du système SGN

Le fichier source `.sgn` est d'abord tokenisé par **Flex**, qui transmet les tokens à **Bison**. Ce dernier construit la structure de données en mémoire, effectue les vérifications sémantiques et déclenche le calcul des moyennes. Le binaire `sgn_parser` émet alors un flux **JSON** sur la sortie standard, lu par l'interface Python via `subprocess.run()`.

2. Spécification du langage `.sgn`

2.1. Description du domaine

Le système gère les notes d'étudiants sur trois niveaux d'études, chacun couvrant deux semestres :

Niveau	Désignation	Semestres	Modules types
L1	Première année de Licence	S1, S2	Algorithmique, Maths, Architecture
L2	Deuxième année de Licence	S3, S4	POO, BDD, Systèmes, Compilation
L3	Troisième année de Licence	S5, S6	IA, Génie Logiciel, Réseaux

Chaque étudiant possède un **matricule unique**, un **nom**, un **prénom** (optionnel), et une liste de modules par semestre, chacun associé à un **coefficient** et une **note** sur 20.

2.2. Structure d'un fichier .sgn

```

1  -- Fichier notes : Annee academique 2025-2026
2  ANNEE "2025-2026"
3
4  NIVEAU L1 {
5      ETUDIANT {
6          MATRICULE : "L1-2025-001"
7          NOM : "DIALLO Mamadou"
8          PRENOM : "Mamadou"
9          SEMESTRE S1 {
10             MODULE "Algorithmique" COEF 3 NOTE 14.5
11             MODULE "Mathematiques" COEF 4 NOTE 12.0
12             MODULE "Architecture" COEF 2 NOTE 16.0
13         }
14         SEMESTRE S2 {
15             MODULE "Programmation C" COEF 3 NOTE 13.5
16             MODULE "Logique" COEF 3 NOTE 11.0
17         }
18     }
19 }
```

Listing 1: Extrait du fichier test_valide.sgn

2.3. Grammaire formelle (BNF)

La grammaire complète du langage .sgn est définie en BNF comme suit :

Grammaire BNF du langage .sgn

```

<programme>      ::= ANNEE STRING <liste_niveaux>
<liste_niveaux>   ::= <niveau> | <liste_niveaux> <niveau>
<niveau>          ::= NIVEAU <id_niveau> '{' <liste_etudiants> '}'
<id_niveau>       ::= L1 | L2 | L3
<liste_etudiants> ::= <etudiant> | <liste_etudiants> <etudiant>
<etudiant>        ::= ETUDIANT '{' <champs_etudiant>
                        <liste_semestres> '}'
<champs_etudiant> ::= MATRICULE ':' STRING NOM ':' STRING
                        [ PRENOM ':' STRING ]
<liste_semestres> ::= <semestre> | <liste_semestres> <semestre>
<semestre>        ::= SEMESTRE <id_sem> '{' <liste_modules> '}'
```

```

<id_sem>      ::= S1 | S2 | S3 | S4 | S5 | S6
<liste_modules> ::= <module> | <liste_modules> <module>
<module>      ::= MODULE STRING COEF ENTIER NOTE REEL
<STRING>      ::= '"' [^"]* '"'
<REEL>        ::= [0-9]+ ('.' [0-9]+)?
<ENTIER>      ::= [0-9]+

```

2.4. Tableau des tokens lexicaux

Token	Description	Exemple
ANNEE	Mot-clé de début de fichier	ANNEE
NIVEAU	Déclaration d'un niveau	NIVEAU
ETUDIANT	Bloc étudiant	ETUDIANT
MATRICULE	Identifiant matricule	MATRICULE
NOM / PRENOM	Champs nominaux	NOM, PRENOM
SEMESTRE	Déclaration d'un semestre	SEMESTRE
MODULE	Déclaration d'un module	MODULE
COEF	Coefficient du module	COEF
NOTE	Note obtenue	NOTE
L1/L2/L3	Identifiants de niveau	L1, L3
S1..S6	Identifiants de semestre	S1, S4
REEL	Nombre réel	14.5, 9.0
ENTIER	Nombre entier	3, 1
STRING_TOK	Chaîne entre guillemets	"Algorithmique"

3. Analyseur lexical — `lexer.1` (Flex)

3.1. Structure du fichier

Le fichier `lexer.1` est organisé en trois sections selon la convention Flex :

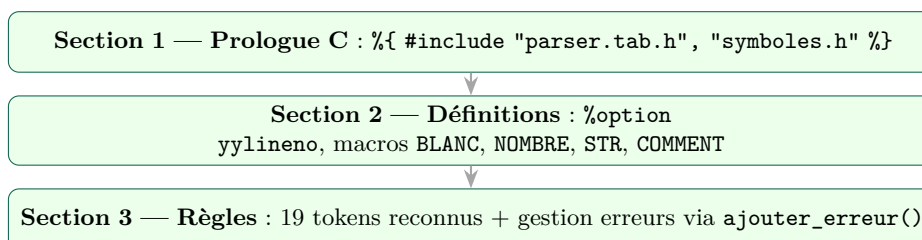


FIG. 2 : Structure du fichier `lexer.1`

Le fichier `lexer.1` est le fichier source de l'analyseur lexical écrit pour Flex. Tout fichier Flex est obliga-

toirement divisé en trois sections séparées par `%`. La première, le **prologue C**, contient du code C pur encadré par `%{` et `%}`, copié tel quel en tête du fichier généré : on y place les `#include` de `parser.tab.h` (qui fournit les numéros de tokens générés par Bison) et de `symboles.h` (qui donne accès aux structures de données et à `ajouter_erreur()`). La deuxième section contient les **définitions** : l'option `%option yylineno` demande à Flex de compter automatiquement les lignes dans `yylineno`, et les macros (`BLANC`, `NOMBRE`, `STR`, `COMMENT`) sont des abréviations d'expressions régulières réutilisables dans la section suivante.

La troisième section, le **cœur du lexer**, contient les règles : chacune associe un motif (expression régulière) à une action (code C). Les 19 règles couvrent les mots-clés (`ANNEE`, `NIVEAU`...), les identifiants de niveau et de semestre (`L1`, `S1`...), les littéraux (`STRING`, `REEL`, `ENTIER`) et les séparateurs (`{`, `}`, `:`). Les deux dernières règles sont dédiées à la gestion des erreurs : elles capturent tout caractère ou chaîne non reconnu et appellent `ajouter_erreur("LEX", yylineno, msg)` pour enregistrer l'erreur sans interrompre le parsing. Flex lit le fichier `.sgn` caractère par caractère, applique ces règles et transmet les tokens reconnus à Bison.

3.2. Options et définitions de macros

```

1 %option noyywrap yylineno warn noinput nounput
2
3 BLANC      [ \t\r]+
4 NL         \n
5 NOMBRE     [0-9]+(\.[0-9]+)?
6 STR        \"[^\n]*\"
7 COMMENT    \"--\"[^\n]*

```

Listing 2: Options et macros dans `lexer.l`

L'option `yylineno` délègue à Flex le suivi automatique du numéro de ligne, indispensable pour la localisation des erreurs. L'option `noyywrap` supprime la redéfinition de `yywrap()`.

3.3. Règles de reconnaissance

```

1 {COMMENT}  { /* commentaire ignore */ }
2 {BLANC}    { /* blancs ignores      */ }
3 {NL}       { /* saut de ligne       */ }
4
5 "ANNEE"    { return ANNEE;          }
6 "NIVEAU"   { return NIVEAU_KW;      }
7 "ETUDIANT" { return ETUDIANT_KW;    }
8 "MODULE"   { return MODULE_KW;     }
9 "COEF"     { return COEF_KW;       }
10 "NOTE"    { return NOTE_KW;       }
11 "L1"      { return NIV_L1_TOK;    }
12 "S1"      { return SEM_S1_TOK;    }
13
14 {NOMBRE}   {
15     if (strchr(yytext, '.')) {
16         yylval.reel = atof(yytext); return REEL;
17     } else {
18         yylval.entier = atoi(yytext); return ENTIER;
19     }
20 }
21 {STR}      { yylval.chaine = strip_quotes(yytext); return STRING_TOK; }

```

Listing 3: Extrait des règles lexicales — mots-clés et littéraux

La règle **NOMBRE** dispatche en deux tokens distincts : **REEL** si le texte contient un point décimal, **ENTIER** sinon. La fonction `strip_quotes()` retire les guillemets et retourne une chaîne allouée dynamiquement avec `malloc()`, libérée dans les actions Bison.

3.4. Gestion des erreurs lexicales

```

1  \["^\n]*\n {
2      char msg[MAX_STR];
3      snprintf(msg, MAX_STR,
4          "chaîne non terminée à la ligne %d", yylineno);
5      ajouter_erreur("LEX", yylineno, msg);
6  }
7  . {
8      char msg[MAX_STR];
9      snprintf(msg, MAX_STR,
10         "caractère illegal '%s' (ASCII %d)",
11         yytext, (unsigned char)yytext[0]);
12     ajouter_erreur("LEX", yylineno, msg);
13 }
```

Listing 4: Collecte des erreurs lexicales dans `g_erreurs[]`

Toute erreur lexicale appelle `ajouter_erreur("LEX", ...)` qui l'enregistre dans le tableau global `g_erreurs[]` et la signale sur `stderr`. Le parsing peut se poursuivre malgré l'erreur.

4. Analyseur syntaxique et sémantique — `parser.y` (Bison)

4.1. Déclarations et union de valeurs

```

1  %define parse.error verbose
2
3  %union { double reel; int entier; char *chaîne; }
4
5  %token ANNEE NIVEAU_KW ETUDIANT_KW MATRICULE_KW NOM_KW PRENOM_KW
6  %token SEMESTRE_KW MODULE_KW COEF_KW NOTE_KW
7  %token NIV_L1_TOK NIV_L2_TOK NIV_L3_TOK
8  %token SEM_S1_TOK SEM_S2_TOK SEM_S3_TOK SEM_S4_TOK SEM_S5_TOK SEM_S6_TOK
9  %token <reel> REEL
10 %token <entier> ENTIER
11 %token <chaîne> STRING_TOK
12 %type <entier> id_niveau id_sem
```

Listing 5: Déclarations Bison — union et tokens

L'union `%union` définit les trois types de valeurs sémantiques possibles. La directive `%define parse.error verbose` produit des messages d'erreur syntaxique détaillés, intégrés dans le JSON de sortie.

4.2. Variables de contexte

Quatre variables statiques maintiennent le contexte de construction pendant le parsing :

```

1  static Niveau      *ctx_niv      = NULL; /* niveau courant */
2  static Etudiant    *ctx_etu      = NULL; /* etudiant courant */
3  static Semestre    *ctx_sem      = NULL; /* semestre courant */
4  static CodeNiveau  ctx_code_niv = 0;    /* code du niveau */

```

Listing 6: Variables de contexte dans `parser.y`

4.3. Règles de production principales

```

1  programme
2      : ANNEE STRING_TOK liste_niveaux
3      {
4          strncpy(g_programme.annee, $2, MAX_STR - 1);
5          free($2);
6          programme_calcule_tout(&g_programme);
7      }
8      ;
9
10 module_item
11     : MODULE_KW STRING_TOK COEF_KW ENTIER NOTE_KW REEL
12     {
13         char msg[MAX_STR];
14         /* Verification 1 : note dans [0,20] */
15         if ($6 < 0.0 || $6 > 20.0) {
16             snprintf(msg, MAX_STR,
17                 "note %.2f hors [0,20] pour \"%s\"", $6, $2);
18             ajouter_erreur("SEM", yylineno, msg);
19         }
20         /* Verification 2 : coefficient >= 1 */
21         if ($4 < 1) {
22             snprintf(msg, MAX_STR,
23                 "coefficient %d invalide pour \"%s\"", $4, $2);
24             ajouter_erreur("SEM", yylineno, msg);
25         }
26         if (ctx_sem) semestre_add_module(ctx_sem, $2, $4, $6);
27         free($2);
28     }
29     ;

```

Listing 7: Règle `programme` et règle `module_item`

4.4. Les quatre vérifications sémantiques

#	Condition vérifiée	Action en cas d'erreur
1	Note $\in [0.0, 20.0]$	ajouter_erreur("SEM", ...) + parsing continue
2	Coefficient ≥ 1	ajouter_erreur("SEM", ...) + parsing continue
3	Matricule unique par niveau	ajouter_erreur("SEM", ...) + ctx_etu = NULL
4	Semestre cohérent avec niveau	ajouter_erreur("SEM", ...) + semestre ignoré

La vérification 4 repose sur la fonction `semestre_valide_pour_niveau()` définie dans `main.c` : L1 n'accepte que S1/S2, L2 que S3/S4, L3 que S5/S6.

4.5. Organigramme du traitement d'une erreur sémantique

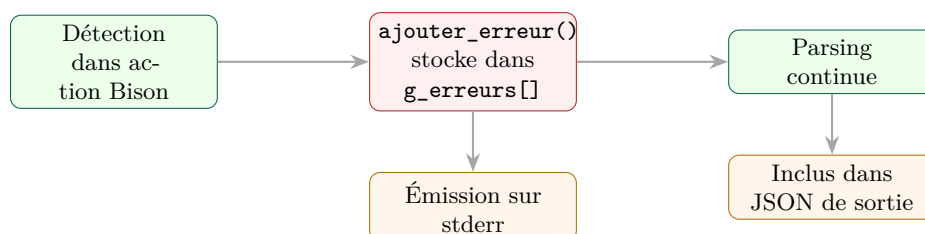


FIG. 3 : Flux de traitement d'une erreur sémantique

La fonction `yyerror()` collecte de même les erreurs syntaxiques :

```

1 void yyerror(const char *msg)
2 {
3     ajouter_erreur("SYN", yylineno, msg);
4 }

```

Listing 8: Collecte unifiée dans `yyerror()`

5. Implémentation C — `main.c` et `symboles.h`

5.1. Architecture des fichiers source

Le fichier `symboles.h` ne contient **aucune implémentation** : uniquement les déclarations de structures, les constantes, les `extern` des variables globales et les prototypes de fonctions. Toutes les implémentations résident dans `main.c`.

5.2. Structures de données

```

1 typedef struct {
2     char    nom[MAX_STR]; int coef; double note;
3 } Module;
4
5 typedef struct {
6     CodeSemestre id;
7     Module        modules[MAX_MOD]; /* MAX_MOD = 64 */

```

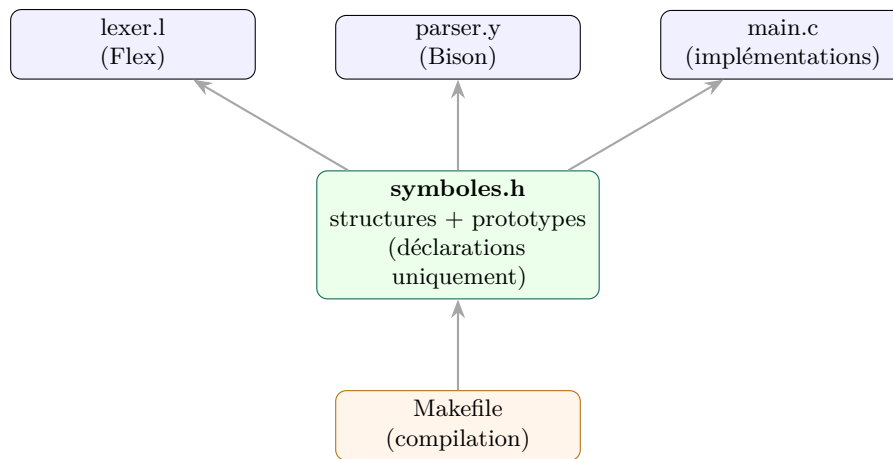


FIG. 4 : Dépendances entre les fichiers source

```

8      int          nb_modules;
9      double       moyenne;
10 } Semestre;
11
12 typedef struct {
13     char          matricule[MAX_STR], nom[MAX_STR];
14     Semestre semestres[MAX_SEM]; /* MAX_SEM = 6 */
15     int           nb_semestres;
16     double        moyenne_annuelle;
17     char          mention[32], decision[16];
18     int           rang;
19 } Etudiant;
20
21 typedef struct {
22     CodeNiveau id;
23     Etudiant   etudiants[MAX_ETU]; /* MAX_ETU = 256 */
24     int        nb_etudiants;
25 } Niveau;
26
27 typedef struct {
28     char   annee[MAX_STR];
29     Niveau niveaux[MAX_NIV]; /* MAX_NIV = 3 */
30     int    nb_niveaux;
31 } Programme;
32
33 typedef struct {
34     int ligne; char type[16]; char message[MAX_STR];
35 } ErreurSGN;
  
```

Listing 9: Hiérarchie des structures dans `symboles.h`

Le choix des **tableaux statiques** (plutôt que l'allocation dynamique) simplifie la gestion mémoire : aucun `free()` nécessaire sur les structures, les limites `MAX_*` sont suffisamment larges pour le contexte universitaire visé.

5.3. Variables globales

```

1 Programme g_programme;           /* arbre de donnees principal */
2 ErreurSGN g_erreurs[MAX_ERR];    /* toutes erreurs LEX/SYN/SEM */
3 int       g_nb_erreurs = 0;       /* compteur d'erreurs          */

```

Listing 10: Variables globales définies dans `main.c`

Ces variables sont accessibles depuis `lexer.l` et `parser.y` grâce aux déclarations `extern` dans `symboles.h`. Ce choix est justifié par la nature du pipeline Flex/Bison : les actions lexicales et syntaxiques doivent pouvoir alimenter la structure globale sans passer de pointeurs à travers la pile LALR.

5.4. Catalogue des fonctions implémentées

Fonction	Rôle
<code>programme_init(p)</code>	Initialise la structure <code>Programme</code> à zéro (<code>memset</code>)
<code>programme_add_niveau(p, code)</code>	Recherche ou crée le niveau <code>code</code> dans <code>p</code>
<code>niveau_find_etudiant(niv, mat)</code>	Recherche linéaire par matricule dans le niveau
<code>niveau_add_etudiant(niv, mat, nom)</code>	Alloue un nouvel étudiant dans le niveau
<code>etudiant_add_semestre(e, id)</code>	Crée un semestre pour l'étudiant courant
<code>semestre_add_module(s, nom, coef, note)</code>	Ajoute un module au semestre courant
<code>programme_calcule_tout(p)</code>	Orchestre les calculs post-parsing
<code>semestre_valide_pour_niveau(sem, niv)</code>	Vérifie la cohérence semestre/niveau
<code>ajouter_erreur(type, ligne, msg)</code>	Enregistre toute erreur LEX/SYN/SEM
<code>programme_to_json(p, out)</code>	Séréalise la structure complète en JSON

5.5. Calculs — formules du sujet

5.5.1. Moyenne pondérée d'un semestre (formule 1)

```

1 static double semestre_moyenne(Semestre *s)
2 {
3     double somme_pond = 0.0, somme_coef = 0.0;
4     int i;
5     for (i = 0; i < s->nb_modules; i++) {
6         somme_pond += s->modules[i].coef * s->modules[i].note;
7         somme_coef += s->modules[i].coef;
8     }
9     return somme_coef > 0.0 ? somme_pond / somme_coef : 0.0;
10 }

```

Listing 11: Implémentation de la formule (1)

$$\bar{M}_s = \frac{\sum_{i=1}^n c_i \times n_i}{\sum_{i=1}^n c_i}$$

5.5.2. Moyenne annuelle (formule 2)

```

1 static double etudiant_moyenne_annuelle(Etudiant *e)
2 {
3     double total = 0.0;
4     int i;
5     if (e->nb_semestres == 0) return 0.0;
6     for (i = 0; i < e->nb_semestres; i++)
7         total += e->semestres[i].moyenne;
8     return total / e->nb_semestres;
9 }

```

Listing 12: Implémentation de la formule (2)

$$\bar{M}_a = \frac{1}{S} \sum_{s=1}^S M_s$$

5.5.3. Décision et mention

```

1 static void etudiant_set_mention(Etudiant *e)
2 {
3     double m = e->moyenne_annuelle;
4     if (m >= 16.0) { strcpy(e->decision, "Admis"); strcpy(e->mention, "Tres
        Bien"); }
5     else if (m >= 14.0) { strcpy(e->decision, "Admis"); strcpy(e->mention, "Bien")
        ; }
6     else if (m >= 12.0) { strcpy(e->decision, "Admis"); strcpy(e->mention, "Assez
        Bien"); }
7     else if (m >= 10.0) { strcpy(e->decision, "Admis"); strcpy(e->mention, "
        Passable"); }
8     else { strcpy(e->decision, "Ajourne"); strcpy(e->mention, ""); }
9 }

```

Listing 13: Attribution de la mention

5.5.4. Tri et calcul des rangs

`niveau_calcule_rangs()` trie le tableau `etudiants[]` par ordre décroissant de `moyenne_annuelle` via un tri à bulles, puis affecte le rang `i+1` à l'étudiant à la position `i`.

5.6. Organigramme de main()

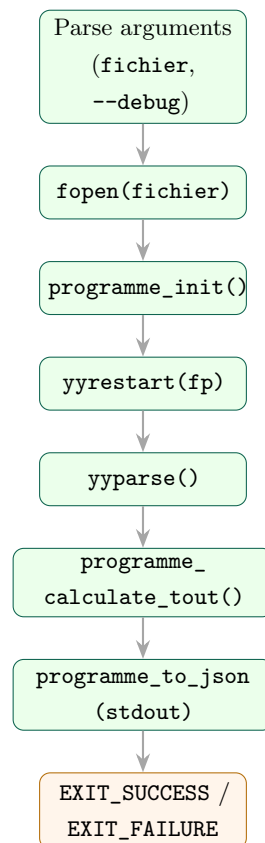


FIG. 5 : Organigramme de la fonction `main()`

5.7. Sérialisation JSON

La fonction `programme_to_json()` parcourt la hiérarchie complète `Programme` → `Niveau` → `Etudiant` → `Semestre` → `Module` et produit le JSON conforme au format §3.3.2 du sujet. La fonction auxiliaire `json_esc()` échappe les caractères spéciaux (" et \) pour garantir un JSON valide même si les noms contiennent des guillemets.

Exemple de sortie JSON pour `test_valide.sgn`

```
{
  "annee": "2025-2026",
  "niveaux": [
    {
      "niveau": "L1",
      "etudiants": [
        {
          "matricule": "L1-2025-001",
          "nom": "DIALLO Mamadou",
          "moyenne_annuelle": 13.64,
          "mention": "Assez Bien",
          "decision": "Admis",
          "rang": 1
        }
      ]
    }
  ]
}
```

```
    }
  ]
}
],
"erreurs": []
}
```

6. Compilation — Makefile

6.1. Compatibilité multi-plateformes

Le Makefile détecte automatiquement le système d'exploitation via `$(shell uname)` et adapte les flags en conséquence :

```
1  OS := $(shell uname)
2
3  ifeq ($(OS), Darwin)          # macOS
4      BREW_BISON := $(shell brew --prefix bison 2>/dev/null)/bin/bison
5      ifeq ($(shell test -x "$(BREW_BISON)" && echo yes), yes)
6          BISON = $(BREW_BISON)
7          BISON_WARN = -Wall    # bison Homebrew recent
8      else
9          BISON = bison
10         BISON_WARN =          # bison Xcode (vieux, sans -Wall)
11     endif
12     LDFLAGS = -ll -lm
13 else                            # Linux (Ubuntu/Debian)
14     BISON = bison
15     BISON_WARN = -Wall
16     LDFLAGS = -lfl -lm
17 endif
```

Listing 14: Détection OS dans le Makefile

6.2. Chaîne de compilation

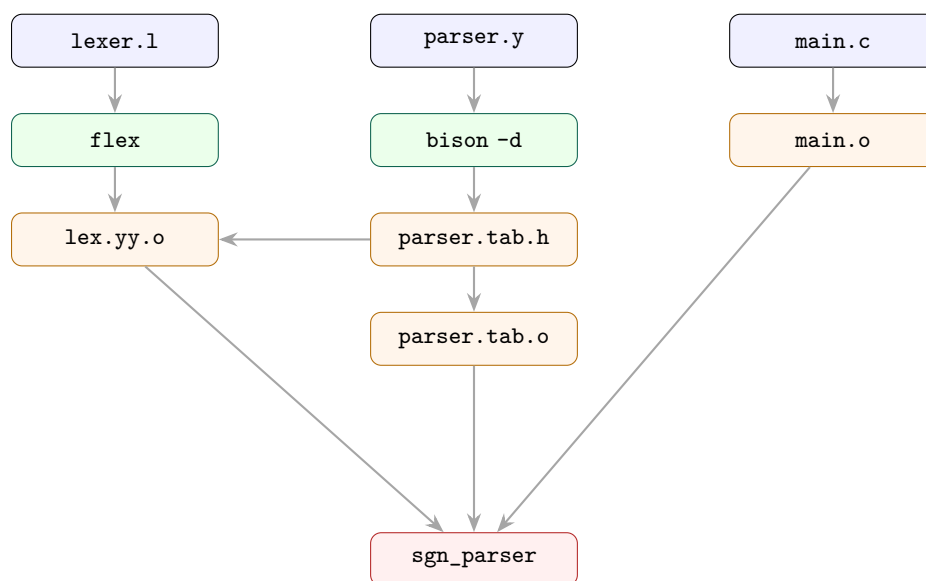


FIG. 6 : Chaîne de compilation du projet SGN

7. Interface graphique Python

7.1. Architecture des fichiers

L'interface est répartie en trois fichiers Python :

Fichier	Rôle
app.py	Fenêtre principale PyQt6 : Vue + Contrôleur
parser_bridge.py	Pont subprocess vers <code>sgn_parser</code> , parse le JSON retourné
interface.py	Constantes de style, widget <code>FilePanel</code> (drag & drop)

7.2. Diagramme de séquence : analyse d'un fichier

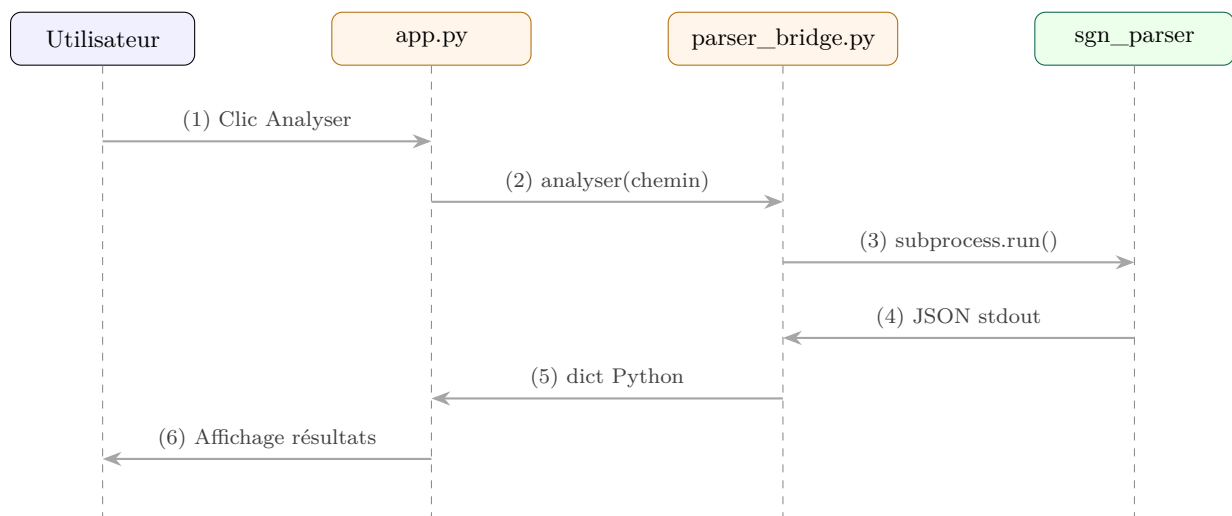


FIG. 7 : Diagramme de séquence : analyse d'un fichier `.sgn`

7.3. Communication subprocess

```

1 res = subprocess.run(
2     [bin_path, chemin_sgn],
3     capture_output=True, text=True, timeout=15
4 )
5 if res.stdout.strip():
6     return json.loads(res.stdout)
  
```

Listing 15: Extrait de `parser_bridge.py` — appel subprocess

La communication entre Python et le binaire C est **unidirectionnelle** : le binaire lit le fichier `.sgn` depuis le disque, écrit le JSON sur `stdout` et les messages de débogage sur `stderr`. Python capture `stdout` via `capture_output=True` et décode le JSON avec `json.loads()`.

7.4. Fonctionnalités de l'interface

L'interface offre les fonctionnalités suivantes :

- **Chargement** par bouton Parcourir ou drag & drop sur le panneau fichier
- **Éditeur intégré** : zone de texte éditable avant analyse, sauvegardée avant chaque appel
- **Onglet Résultats** : tableau PyQt6 avec colonnes Matricule, Nom, Niveau, Semestre, Moyennes, Rang, Mention, Décision
- **Onglet JSON brut** : affichage du JSON complet retourné par `sgn_parser`
- **Bandeau d'erreurs** : apparaît uniquement si des erreurs sont détectées, avec type (LEX/SYN/-SEM), numéro de ligne et message
- **Export CSV** : tableau des résultats + section erreurs en bas
- **Export TXT** : tableau ASCII avec colonnes centrées + section erreurs

8. Tests et validation

8.1. Stratégie de tests

Quatre fichiers de test couvrent les cas nominaux et d'erreur :

Fichier	Type	Contenu
test_valide.sgn	Nominal	3 étudiants sur 2 niveaux, toutes données correctes
test_erreur_lex.sgn	Erreur LEX	Caractère illégal #, chaîne non terminée
test_erreur_syn.sgn	Erreur SYN	Token manquant, accolade non fermée
test_erreur_sem.sgn	Erreur SEM	Note 22.0, coef 0, doublon matricule, S3 dans L1

8.2. Vérification de la compilation

Sortie de make sur Ubuntu 24.04

```
apple@ubuntu:/Users/Apple/Downloads/Papa-Mbaye-BA_Gilbert-Ngor-Thiomby-DIOUF_SGN_M1/sgn_project/src$ make
bison -d -Wall parser.y
gcc -Wall -Wextra -g -std=c99 -D_POSIX_C_SOURCE=200809L -c parser.tab.c
flex lexer.l
gcc -Wall -Wextra -g -std=c99 -D_POSIX_C_SOURCE=200809L -Wno-sign-compare -c lex.yy.c
gcc -Wall -Wextra -g -std=c99 -D_POSIX_C_SOURCE=200809L -c main.c
gcc -Wall -Wextra -g -std=c99 -D_POSIX_C_SOURCE=200809L -o sgn_parser parser.tab.o lex.yy.o main.o -lfl -lm
✓ Compilation réussie → sgn_parser
Usage : ./sgn_parser <fichier.sgn> [--debug]
✓ sgn_parser copié dans gui/
```

8.3. Résultats obtenus

8.3.1. Test 1 — Fichier valide

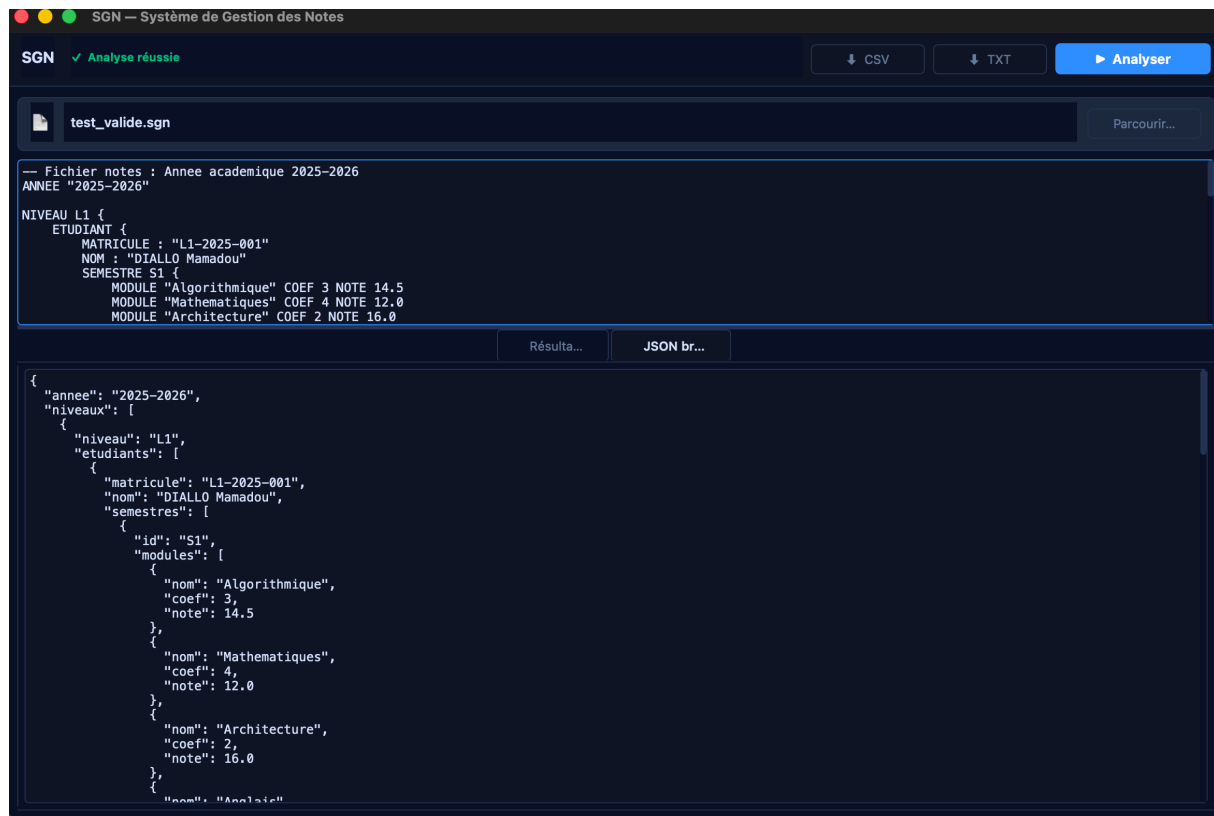


FIG. 8 : Test d'un fichier .sgn valide

8.3.2. Test 4 — Erreurs sémantiques

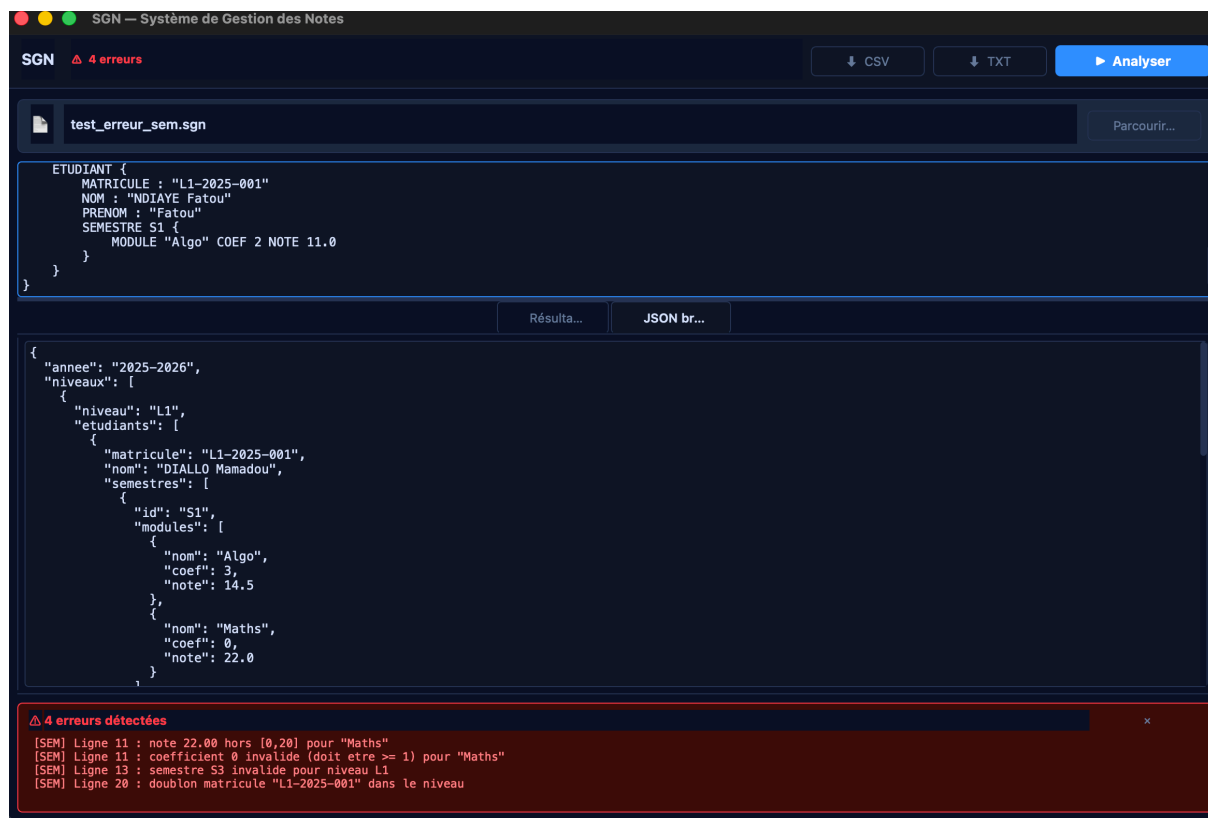


FIG. 9 : Test d'un fichier .sgn contenant des erreurs sémantiques

Point clé : le JSON est **toujours émis**, même en présence d'erreurs. Les erreurs apparaissent dans le tableau "erreurs":[...] du JSON, rendant l'interface Python opérationnelle dans tous les cas.

9. Conclusion

9.1. Bilan des fonctionnalités

Fonctionnalité	Réalisé
Analyseur lexical Flex (tous tokens, erreurs, commentaires)	✓
Analyseur syntaxique Bison (grammaire BNF conforme)	✓
Construction table/structure de données	✓
Vérifications sémantiques (4 règles)	✓
Calculs moyennes, rangs, mentions	✓
Interface graphique PyQt6 (chargement, édition, tableau)	✓
Affichage erreurs coloré avec numéro de ligne	✓
Export CSV et TXT	✓
Rapport technique	✓

9.2. Difficultés rencontrées

Les principales difficultés rencontrées ont été les suivantes. La gestion des variables de contexte Bison (`ctx_niv`, `ctx_etu`, `ctx_sem`) a nécessité une attention particulière : une affectation prématurée ou un oubli de réinitialisation à `NULL` provoquait des erreurs de segmentation silencieuses. La collecte unifiée des erreurs LEX, SYN et SEM dans un seul tableau JSON a requis la refactorisation de `yyerror()` et des règles du lexer pour appeler `ajouter_erreur()` de façon cohérente. La compatibilité macOS/Linux au niveau du Makefile a demandé la détection de l'OS via `$(shell uname)` et l'adaptation des flags `-Wall` (non supporté par le bison embarqué dans Xcode) et des bibliothèques de liaison (`-ll` vs `-lf1`).

9.3. Perspectives d'amélioration

Plusieurs améliorations pourraient être envisagées. L'ajout d'un **export PDF** directement depuis l'interface Python permettrait une remise de bulletins plus professionnelle. La prise en charge de **L4** et **L5** nécessiterait uniquement d'étendre les énumérations `CodeNiveau` et `CodeSemestre` et d'ajouter les tokens correspondants dans le lexer. Enfin, une saisie directe dans la GUI (sans passer par un fichier disque intermédiaire) améliorerait l'ergonomie pour les utilisateurs avancés.